From 280 GB of RAM to one GPU.

The Full Story of Fine-Tuning Large Language Models

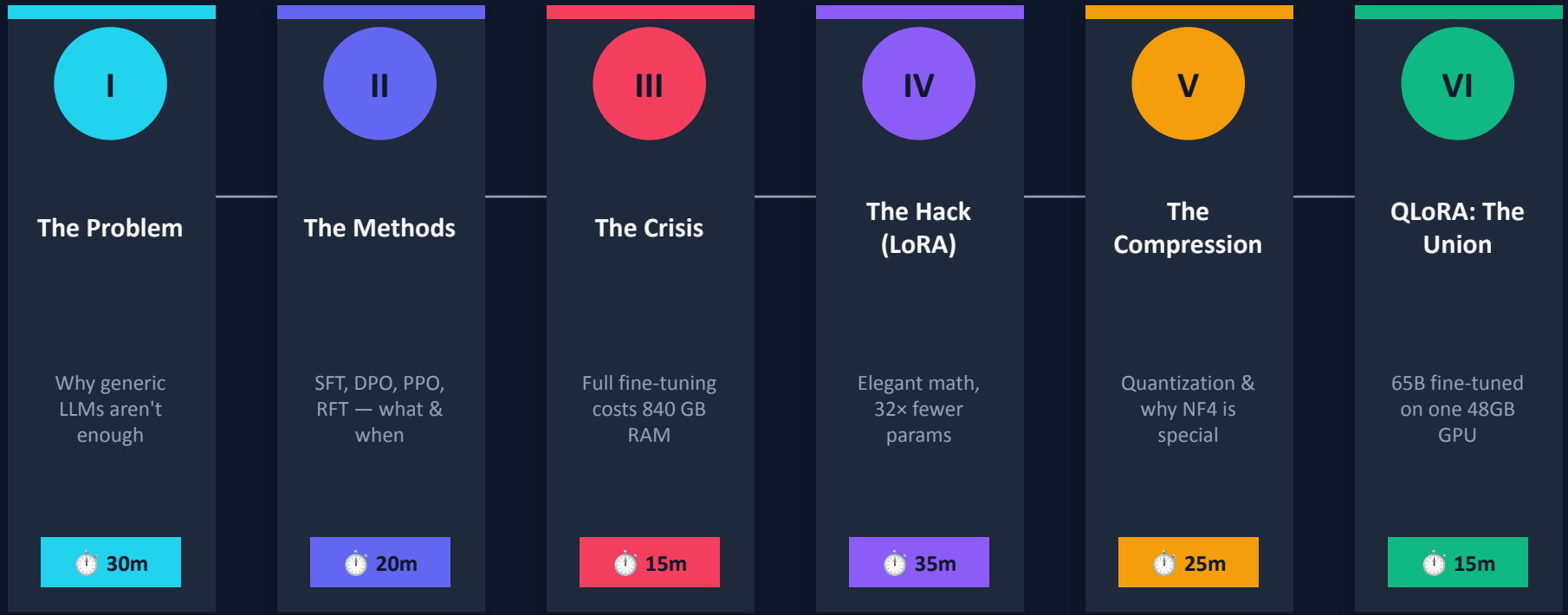
PEFT · LoRA · Quantization · NF4 · QLoRA

3-Hour Session · Sumit Ranjan · AI Security Researcher & Educator

OWASP Agentic AI Top 10 Contributor

Our Journey Today

A story in 6 acts. Each act solves a problem the previous act creates.



Every concept answers a 'why' before it answers a 'what'.

ACT I

The Problem

Why isn't a generic LLM good enough?

By the end of this act, you'll know exactly when fine-tuning is — and isn't — the answer.

🗓️ Day 1 at your new company.

Your boss says:
"Our AI gives wrong answers. Fix it."

You have three tools in your belt:

✍️ **Prompt Engineering**

📚 **RAG**

🎛️ **Fine-Tuning**

✋ YOUR TURN

What do you try first — and why?

Commit to one answer before the next slide. No peeking.

- Option A — Prompt Engineering: Craft a better system prompt
- Option B — RAG: Connect the model to your company's knowledge base
- Option C — Fine-Tuning: Train the model on your domain data
- What information would you need to make this decision?

⏱️ 90 sec

The Three Tools — What Each Actually Does

DECISION LOGIC

✍️ Prompt Engineering

When: Try FIRST. Always.

👍 Strengths

- Zero cost, zero infra
- Instant to test and iterate
- Works for style, format, instruction-following

👎 Weaknesses

- Fragile for deep domain knowledge
- Fails when model simply doesn't know the facts
- No persistent memory or persona

✅ **Always try this before anything else**

📚 Retrieval Augmented Generation (RAG)

When: Try SECOND. When knowledge is the gap.

👍 Strengths

- Works with dynamic / frequently changing data
- No re-training required
- Source-citable answers

👎 Weaknesses

- Can't change the model's style or persona
- Missing empathy, tone, specialized reasoning
- Retrieval latency at inference

✅ **Best for knowledge gaps, not style gaps**

🎛️ Fine-Tuning

When: Use when prompting + RAG still fail.

👍 Strengths

- Deeply changes how the model thinks and speaks
- Perfect for style, persona, domain reasoning
- Faster inference (no retrieval step)

👎 Weaknesses

- Expensive to train and maintain
- Needs labeled data
- Model goes stale after training

🎯 **Use for: style, persona, deep domain expertise**

See the Difference: RAG vs Fine-Tuned Response

REAL EXAMPLE

Prompt: "My father was just diagnosed with early-stage Alzheimer's. What should I do?"

📚 RAG Response

According to the Alzheimer's Association, early-stage Alzheimer's involves mild memory loss. Recommended steps include:

- Consult a neurologist for a formal diagnosis
- Begin legal and financial planning
- Research memory care facilities
- Consider medication options such as cholinesterase inhibitors

For more information, visit alzfdn.org.

Factually correct. Emotionally sterile.

V
S

🎛️ Fine-Tuned Response (Empathetic Medical AI)

I'm so sorry to hear about your father. This is an incredibly difficult time, and it's okay to feel overwhelmed.

Here's what matters most right now:

First, take a breath. You don't have to figure everything out today.

The most helpful immediate step is a visit to a geriatric neurologist — they'll assess the stage and give you a clear picture of what lies ahead.

You might also want to start a simple journal of daily changes. It helps doctors enormously.

You're already doing the right thing by asking.

Same facts. Completely different experience.

ACT II

The Methods

You've decided to fine-tune. But there are four very different ways to do it. Which one is right depends on the data you have.

STOP & THINK · MYSTERY 1

You're fine-tuning a 7B model. Should you start from the BASE model or the INSTRUCT model?

Think about what you want to change — the knowledge, or the behaviour?

Base Model vs Instruct Model — The DNA Difference

🏗️ Base Model (e.g. Llama-3-8B)

Trained to: Predict the next token

Input: "The capital of France is"
Output: "Paris. Paris is also home to the Eiffel Tower, which was built in..."

It completes text — it does NOT answer questions.

Use when: You want to teach a completely new language, domain, or output format from scratch.

⚠️ Risk: Without instruction tuning, it may not follow your prompts reliably.

🤖 Instruct Model (e.g. Llama-3-8B-Instruct)

Trained to: Follow instructions helpfully

Input: "What is the capital of France?"
Output: "Paris."

Already knows how to chat, answer, and refuse harmful requests.

Use when: You want to specialise the model's persona, style, or domain knowledge — while keeping its instruction-following intact.

✅ Best starting point for 95% of practical fine-tuning tasks.

Four Methods, One Question Each: How Do You Learn?

METHODS MAP

SFT

❓ **Do you have labeled (prompt, correct answer) pairs?**

→ Show the model examples of right answers. Train it to copy them.

📦 **(prompt → ideal response) pairs**

🎭 *Textbook + answer key*

DPO

❓ **Do you have ranked pairs — a better and worse answer?**

→ Show the model (chosen, rejected) pairs. Train it to prefer the chosen one.

📦 **(prompt, chosen response, rejected response)**

🎭 *Tasting menu — 'I prefer dish A over B'*

PPO

❓ **Can humans rank model outputs in real time?**

→ Train a reward model from human rankings. Use RL to maximise reward.

📦 **Human-ranked responses + reward model**

🎭 *Dog training — reward good behaviour, ignore bad*

RFT

❓ **Can you verify a correct answer programmatically (e.g. run unit tests)?**

→ Model generates answers, verifier scores them (pass/fail). No humans needed.

📦 **Just prompts + a verifiable reward function**

🎭 *Practice problems with a self-grading calculator*

🎯 YOUR TURN

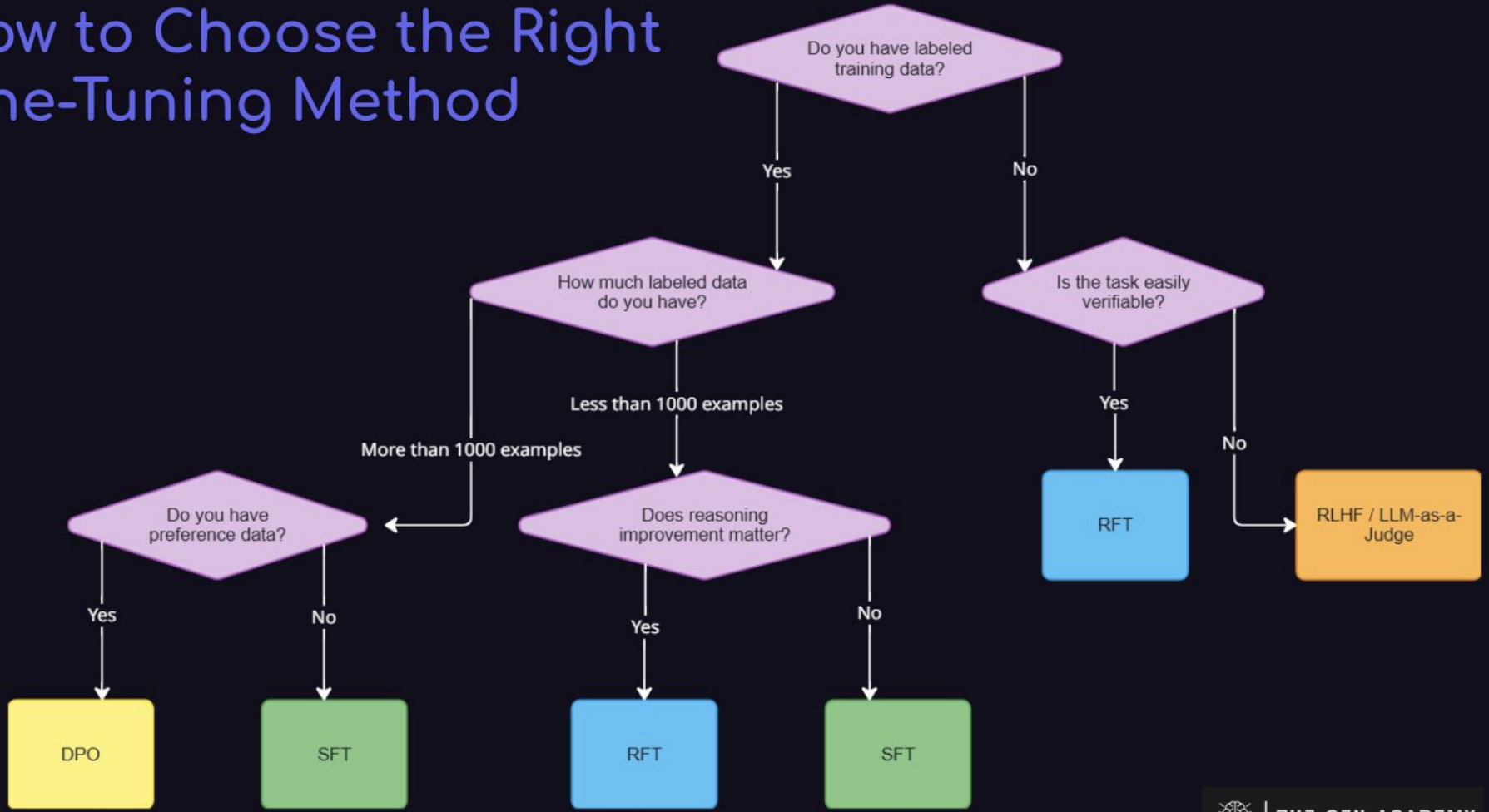
Match the Method to the Task

For each scenario below, call out which method (SFT / DPO / PPO / RFT) you'd use — and why.

- 1️⃣ You have 10,000 customer support transcripts, all labelled with 'good' or 'bad'
- 2️⃣ You're training a code assistant — every answer can be verified by running the code
- 3️⃣ You want a medical chatbot that 'sounds like' a compassionate doctor
- 4️⃣ You have humans ranking AI-written essays but no ground-truth answers

⏱️ 3 min

How to Choose the Right Fine-Tuning Method



⚠️ PLOT TWIST

Full Fine-Tuning costs 840 GB of RAM.

For a 70B model. That's not a typo. Let's find out why.

STOP & THINK · QUICK MATH

A 7B model = 7 billion parameters.
Each parameter = 4 bytes (float32).
How much RAM to train it?

Work it out: 7,000,000,000 × 4 bytes = ? GB

✅ THE ANSWER

7B × 4 bytes = 28 GB just for the weights. And that's before training even starts.

But training also needs gradients (×1) and the Adam optimizer states (×2 more).

Total: 7B params × 12 bytes = 84 GB for the 7B model alone.
For 70B: 840 GB. You'd need 10+ top-of-the-line A100 GPUs.

Most companies — and all individuals — simply can't afford this.

Why Does Training Need 3× the Memory of the Model?

THE OPTIMIZER TAX

Adam optimizer — the gold standard — stores THREE things per parameter:

1

Model Weights (W)

The actual parameters being updated

4 bytes/param (FP32)

2

First Moment (Momentum m)

Running average of past gradients — tells us the direction of update

4 bytes/param (FP32)

3

Second Moment (Velocity v)

Running average of squared gradients — tells us how large the update should be

4 bytes/param (FP32)

4 + 4 + 4 = 12 bytes per parameter · 7B × 12 = 84 GB · 70B × 12 = 840 GB

💭 YOUR TURN

You can't change the 12-bytes-per-param fact. What can you change?

Brainstorm with the person next to you. There are at least 3 levers. What are they?

- Lever 1: What if we only trained SOME of the parameters?
- Lever 2: What if we reduced the precision (bytes per param) of the stored weights?
- Lever 3: What if we used multiple GPUs and split the memory?
- BONUS: What's the trade-off of each lever?

⏱️ 2 min

ACT IV

The Hack (LoRA)

What if you could get 95% of the benefit of full fine-tuning by training less than 1% of the parameters?

ANALOGY FIRST

Don't rewrite the album. Release a remix EP.

Imagine an artist's debut album — 1000 songs, years of work. A new genre is trending. Rewriting all 1000 songs takes years. Instead, they release a 12-song remix EP that clips onto the original. Listeners hear the original + the new flavour. The original album is never touched.

🔗 **Now the real thing: LoRA adds a tiny 'remix EP' of trainable weights on top of the frozen original model.**

STOP & THINK · THE KEY INSIGHT

Fine-tuning changes the weights. But do those changes use the full complexity of the original weight space?

Researchers found that the weight UPDATES (ΔW) during fine-tuning have very low 'rank'. What does that mean for us?

LoRA: The Elegant Matrix Decomposition

THE MATH

Normal fine-tuning updates: $W' = W + \Delta W$ (train all of ΔW — very expensive)

LoRA's insight: ΔW doesn't need full rank. Decompose it.

ΔW ($d \times d$) $512 \times 512 = 262{,}144$ params

$\approx$

A ($d \times r$) $512 \times 8 = 4{,}096$

$\times$

B ($r \times d$) $= 8 \times 512 = 4{,}096$ params

The Savings

Full ΔW: **$512 \times 512 = 262{,}144$ params**

LoRA A+B: **$512 \times 8 + 8 \times 512 = 8{,}192$ params**

Reduction: **32× fewer parameters to train**

At inference: merge $W' = W + AB$
(zero extra latency — free at runtime)

Where LoRA adapters plug in: Wq, Wk, Wv matrices inside every attention layer. Original W stays frozen.

🧮 YOUR TURN

Choose Your Rank (r) — And Justify It

For each scenario, pick a rank and explain your reasoning. No right/wrong — we want your logic.

- 1️⃣ You want a customer support bot to be more polite and concise → r = ?
- 2️⃣ You want a model to reason through multi-step medical diagnoses → r = ?
- 3️⃣ You're resource-constrained (24GB GPU, 7B model, 1000 examples) → r = ?
- Key principle: higher r = more expressiveness, more memory, risk of overfitting
- Paper default: r = 8 · Most practical tasks: r = 4 to 16

⏱️ 2 min

⚠️ NEW PROBLEM

LoRA still needs 140 GB for 70B.

LoRA cut TRAINABLE params by 32×. But the frozen base model still loads in full precision.

ANALOGY FIRST

Quantization = JPEG for Model Weights

A raw photo: 24 MB. Save as JPEG at 80% quality: 2 MB. Is it perfect? No. Can you tell the difference for most purposes? Also no. The eye — like neural networks — doesn't need every decimal place to work well.

🔗 **Now the real thing: We're going to do the same to model weights: trade a tiny bit of precision for a massive drop in memory.**

Original Image

“Quantized” Image

Image adapted from the original by Slava Sidorov.

STOP & THINK · QUICK MATH

Float32 uses 4 bytes per weight.
Int8 uses 1 byte.
If a 70B model needs 280 GB in FP32, how much does it need in Int8?

Calculate: 280 GB × (1/4) = ? · What about Int4?

✅ THE ANSWER

280 GB → 70 GB (Int8) → 35 GB (Int4)

That's a 4× and 8× reduction respectively.

70B in Int4 fits comfortably in a single 48GB A100 GPU.

But here's the catch: equal-width bins destroy information for normally distributed weights. That's where NF4 comes in.

🎯 YOUR TURN

Spot the Mistake — Before We Reveal It

You decide to quantize weights to 4-bit. You use 16 equal-width bins across the range [-8, +8]. Below are two distributions of weight values. Which approach loses more information — and why?

- Distribution A: Uniform — weights spread evenly from -8 to +8
- Distribution B: Normal — 90% of weights clustered between -1 and +1
- THINK: What happens to Distribution B when you use equal-width bins?
- HINT: Most weights fall into 2-3 middle bins. The other 13 bins are nearly empty.

⏱️ 90 sec

LoRA says:

“We do not need to learn the full big matrix directly.”

Instead, we decompose it into two small matrices:

$$\Delta W = A \cdot B$$

Where:

$$A(d, r), \quad B(r, d)$$

Here:

- $d = 512$
- r = rank (a small hyperparameter like 4, 8, 16)

Suppose:

$$r = 8$$

Then:

$A \in \mathbb{R}^{512 \times 8}, \quad B \in \mathbb{R}^{8 \times 512}$

NF4: Why Equal-Width Bins Destroy Neural Network Weights

NF4 DEEP DIVE ①

⚠️ **You flagged NF4 as confusing. Let's build it from first principles.**

❌ Regular int4: 16 Equal-Width Bins

Bin width = constant = (max - min) / 16

Neural network weight distribution:
→ Near-zero: 80%+ of weights
→ Tails (large +/- values): very rare

What happens:
→ Middle 2-3 bins overflow with weights
→ The other 13 bins hold almost nothing
→ Thousands of different weights all map to the SAME quantized value
→ Massive precision loss exactly where you need it most

✅ NF4: 16 Equal-AREA Bins (Quantiles)

Bin size = variable, but each bin holds exactly the same % of weight values:

100% ÷ 16 bins = 6.25% per bin

Near-zero bins: many, very narrow
→ High precision where weights cluster

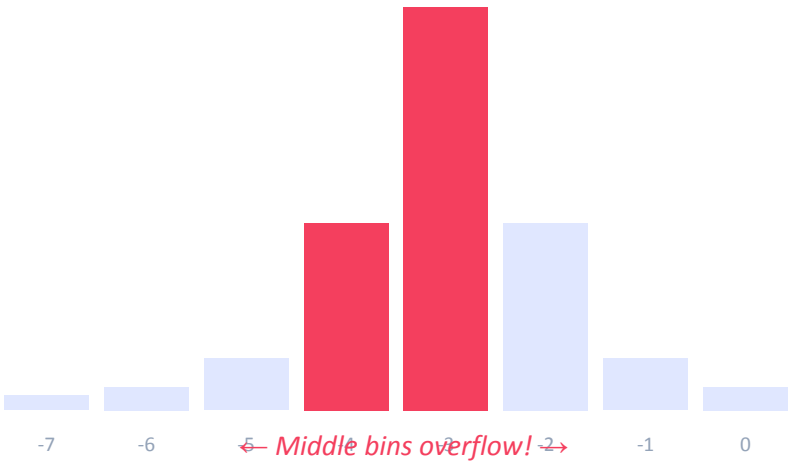
Tail bins: few, very wide
→ Low precision where weights are rare

This is information-theoretically OPTIMAL for data that follows a normal distribution.

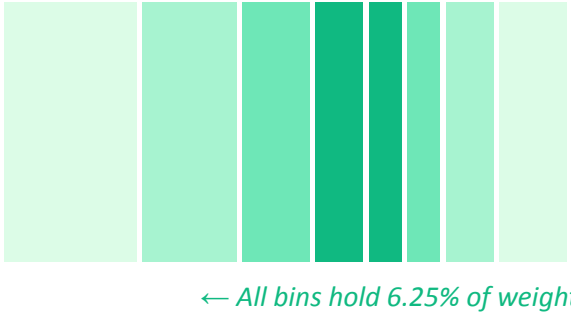
NF4 Visual: Equal Width vs Equal Area

Think about a parking lot built for commuters (most arrive at 9am) vs one built for general use:

❌ Regular int4 — Equal bin widths



✅ NF4 — Equal-area (quantile) bins



NF4 bin boundaries = derived from the inverse CDF of a normal distribution → quantile function → 6.25% per bin

🧮 YOUR TURN

Check the Maths — and Then Go Deeper

Work these out before the next slide reveals the answer.

- 1️⃣ 4-bit = 2^4 = __ possible values = __ bins
- 2️⃣ 100% ÷ 16 bins = __% per bin (this is the NF4 target per bin)
- 3️⃣ 8-bit would give 2^8 = __ bins → __ % per bin
- BONUS: If weights follow N(0,1), which bin boundary lies at the 6.25th percentile? (Hint: use z-table or scipy.stats.norm.ppf(0.0625))

⏱️ 90 sec

✅ THE ANSWER

16 bins · 6.25% each · boundaries from Normal CDF

4 bits ⟶ 2^4 = 16 values. 100 ÷ 16 = 6.25%.

NF4 uses scipy.stats.norm.ppf(i/16) for i = 1..16 to compute each boundary.
The result: very narrow bins near 0 (where 80% of weights live) and wide bins at the tails.

This is why QLoRA paper says NF4 is 'information-theoretically optimal'
for normally distributed data.

ACT VI

QLoRA — The Union

Dettmers et al., 2023. One paper. Three ideas. The result:
Fine-tune a 65B model on a single 48GB GPU.

STOP & THINK · FINAL CHALLENGE

We have NF4 (base model in 4-bit) and LoRA. Problem solved?

What two problems remain?

Hint 1: NF4 needs quantization constants — how much memory do they take?
Hint 2: What happens if the GPU runs out of memory mid-training?

QLoRA: Three Innovations, One Paper

QLORA CORE

① NF4 Quantization

You already know this! Base model weights → 4-bit NF4.
Narrow bins near 0, wide at tails.
LoRA adapters stay in BF16 (full precision where it matters).

📈 280 GB → 35 GB (8× reduction for 70B)

② Double Quantization

NF4 needs one 'scale constant' per 64-weight block.
For 7B model: 7B ÷ 64 = 109M constants × 4 bytes = 437 MB extra.

Solution: Quantize the constants themselves to 8-bit.
Result: Save ~0.37 GB for 7B, much more for 65B+.

📈 Constants: FP32 (437MB) → Int8 (~110MB)

③ Paged Optimizers

Training causes GPU memory spikes during backprop.
If GPU overflows → crash (OOM error).

Solution: Use NVIDIA unified memory.
Optimizer states page out to CPU RAM when GPU fills up.
Page back in when needed — like virtual memory in an OS.

📈 No more OOM crashes on long training runs

🏗️ YOUR TURN

Design a Real QLoRA Config

You have one A100 40GB GPU. Task: fine-tune LLaMA-3 70B on medical consultation transcripts.

- Step 1: What base model variant would you load? (NF4 or BF16?) — and why?
- Step 2: What LoRA rank (r) would you set? Target modules (q_proj, v_proj...)?
- Step 3: What dataset size and format? (SFT with JSONL? How many examples?)
- Step 4: What eval metric tells you it's working vs just memorising?

⏱️ 4 min

The QLoRA Payoff: What Became Possible

RESULTS

📄 **From the paper: 65B model fine-tuned on 1 GPU, matching full fine-tuning quality.**

Method	GPU Required	Memory	Can a startup do this?
Full Fine-Tune (FP32) 70B	10× A100 80GB	840 GB	❌ No
LoRA on BF16 base 70B	2× A100 80GB	140 GB	🟡 Maybe (expensive)
QLoRA — 70B	1× A100 48GB	~35 GB	✅ Yes
QLoRA — 7B	1× RTX 3090 24GB	~6 GB	✅ On a gaming GPU!

🔑 *QLoRA = NF4 quantization of base model + LoRA adapters in BF16 + double quantization + paged optimizers. All four together. That's the recipe.*

EPILOGUE

Tools, Pipeline & Evaluation

You now understand the WHY behind everything.
Here's the HOW in practice.

The Fine-Tuning Pipeline — In Order

PIPELINE

1

Define the eval metric FIRST

Before any training. If you don't know how to measure 'better', you'll never know if you got there.

❌ Trap: Training for 3 days then realising you can't measure success.

2

Curate & clean the dataset

500–50K (prompt, completion) pairs. Remove duplicates. Format to JSONL/ShareGPT. Quality > quantity.

❌ Trap: Poisoned or inconsistent labels make the model worse than baseline.

3

Choose model & PEFT config

Instruct base for most tasks. Set r=8, alpha=16, target modules (q_proj, v_proj). Match seq_len to data.

❌ Trap: Rank too high for a simple task → overfitting on small datasets.

4

Train & monitor loss

Val loss dropping = learning. Val loss rising while train loss falls = overfitting. Stop early.

❌ Trap: Running to 100 epochs without monitoring. Overfitting guaranteed.

5

Evaluate & iterate

Task metric + human eval + regression tests on general capability. Compare to base model.

❌ Trap: Only checking train accuracy. Fine-tuning can destroy general reasoning.

The Toolbox: What to Use and When

ECOSYSTEM

☁️ Cloud Platforms

- HuggingFace AutoTrain — no-code fine-tuning
- Azure OpenAI — GPT fine-tuning with your data
- AWS SageMaker — scalable enterprise pipelines
- Vertex AI — Gemini supervised tuning + RLHF

🛠️ Open Source Frameworks

- Unsloth — 2× faster, 60% less memory, use this first
- TRL (HuggingFace) — SFT, DPO, PPO, GRPO, all in one
- Axolotl — YAML config, reproducible, great for teams
- DeepSpeed / FSDP — multi-GPU distributed training

📊 Evaluation Tools

- Weights & Biases — loss curves, experiment tracking
- LM Eval Harness — standard LLM benchmarks
- RAGAS — hallucination & faithfulness for RAG
- LLM-as-Judge (GPT-4 / Claude) — scale human eval

📦 Datasets to Know

- Alpaca — 52K instruction-following pairs
- OpenHermes — high-quality, diverse instruction data
- Medical Meadow — clinical fine-tuning benchmark
- ShareGPT format — conversational multi-turn data

The Story in 6 Beats

Act I	Prompt first. RAG second. Fine-tune only when the gap is in style, persona, or deep domain reasoning.
Act II	The method depends on your data. Labels → SFT. Pairs → DPO. Rankings → PPO. Verifiable outputs → RFT.
Act III	Full fine-tuning costs 12 bytes/param (weights + gradients + optimizer). That's 840GB for 70B. PEFT is the answer.
Act IV	LoRA decomposes ΔW into $A \times B$. Start with r=8. 32× fewer parameters to train. Zero inference overhead.
Act V	NF4 = quantile-based 4-bit. 6.25% per bin. Optimal for normal distributions. NOT equal-width bins.
Act VI	QLoRA = NF4 base + BF16 LoRA + double quantization + paged optimizers. Fine-tune 65B on 48GB. It works.

Fine Tune Team:
Group 1: Divya, **Abe**, Ankit, Harsh, Chirag
Group 2: Chetan, Zenil, **Yash Gupta, Yash Pramar**

RAG Team:
Group 1: Om, **Sharanya**, Vishal
Group 2: Shanmuk, Avik, **Hiten**

Voice Team:
Group 1: Avishka, Aditya, Himkar, **Het**, Harshit
Group 2: Anvay, Niranjan, **Ayush Patil,** Sudhanshu
Group 3: **Shubham**, Anirudh, Bryson, Aditi

📚 Go Deeper

Papers

- LoRA (Hu et al., 2021) — arxiv.org/abs/2106.09685
- QLoRA (Dettmers et al., 2023) — arxiv.org/abs/2305.14314
- DPO (Rafailov et al., 2023) — arxiv.org/abs/2305.18290

Code

- github.com/unslothai/unsloth
- github.com/huggingface/peft
- github.com/huggingface/trl
- github.com/hiyouga/LLaMA-Factory

Learning Path

- mlabonne/llm-course on GitHub
- HuggingFace NLP Course (free)
- OWASP Agentic AI Top 10 (for AI security context)

Now you know not just WHAT LoRA and NF4 do — but WHY they had to be invented.

Open Q&A — Ask anything.